

Monitoring Server, Applications

A lot of monitoring solutions are available in market today with integration to different data sources.

Some of the popular monitoring tools are ELK, Prometheus, Grafana, Nagios, Datadog, etc. The basic monitoring metrics needs to be collected by the monitoring agent. For some of the system information we can use indigenous tools like SAR, TOP, FREE, DF etc... as and when needed.

Which metrics are interesting to monitor?

- Server & Application UP/DOWN times.
- CPU: Idle, busy, User, IOWait etc.
- Load: 1M, 5M, 15M.
- Memory: Total, Used, Free, Swap total, Swap used, Swap free.
- Disk & Partition: Available space, READS, WRITES per sec, Average IO Utilization.
- Volume: Total, Free, Used.
- Monitoring agent heartbeat to know or understand how available the server is.
- Network: [Incoming & Outgoing] [packets and errors] per second, Throughput on each network interface. Size of incoming & outgoing data per second.
- Process: Per process or process class utilization of CPU, memory, threads.
- Application/Proxy server response time, Cache Hits/Miss, Active connections, Status Codes, User agents.
- Requests [received, handled, failed, processed, denied] per second.
- Certificate Expiry.
- Maximum, Minimum & Average requests count, etc.

Why do we need to monitor SSL certificate expiry?

We should ensure that the SSL certificates need to be up to date as the expiry of it could lead to greater problems like stopping of some important applications or services in an ecosystem which is a security risk and directly impacts business. When we have too many certificates to be monitored it's a tedious job to manually do it. Hence having a dashboard to manage and monitor it centrally is very much needed and alert the security admin in case the expiry of a certificate is nearing.

How would you monitor those metrics?

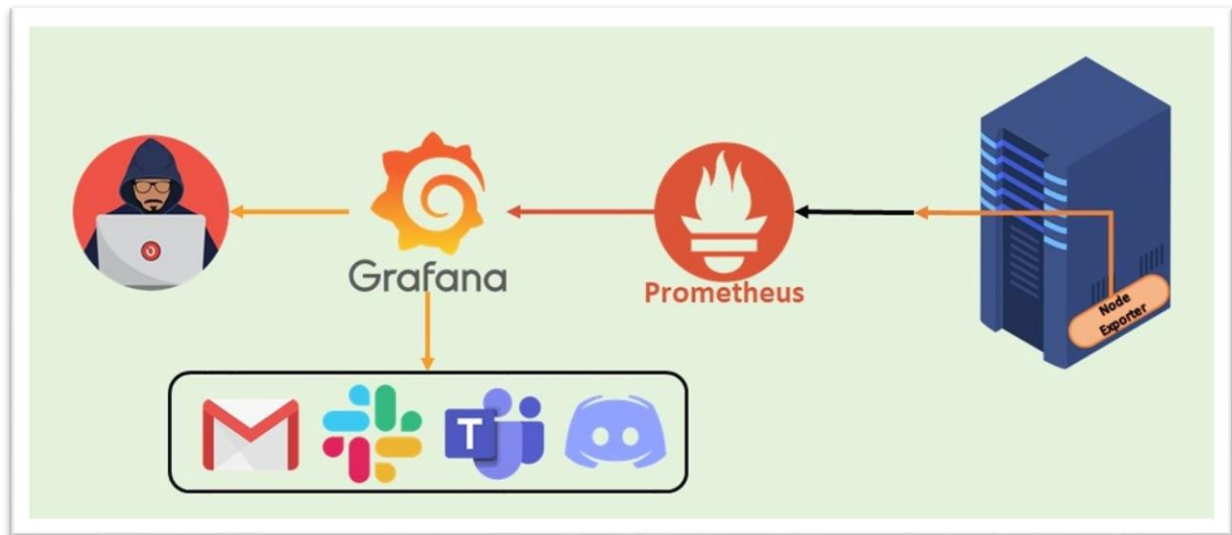
The best way to monitor all the system and application metrics is to unify it all in a dashboard or categorize it based on the type of the metric we plan on viewing it. To visualize the resource usage, a time interval can be selected to collect these performance metrics of a specific server or a cluster of nodes.

Prometheus is a "Pull" based system, meaning the Prometheus server fetches (scrapes) the metrics from the running application periodically. We can configure node exporter inside the node and let Prometheus scrape the metrics exposed.

More on how to setup Grafana & Prometheus on a Linux server can be read [here](#).

Please note that the link contains the steps to setup the monitoring solution on Ubuntu system. However, there are wide variety of guides available online for different distributions.

A simple flow on monitoring.



Monitoring Challenges / Things to consider.

- Choosing a best monitoring agent that doesn't impact the system's performance overall. Possibility to monitor the usage of the monitoring agent would be a great plus indeed. Prometheus has this option to analyze its own metrics.
- Configuring proper retention and backup policies.
- Configuring thresholds for the important parameters to ensure the system is always up and running.
- Triggering instant notifications by setting up alert channels when a particular parameter reaches its threshold or has crossed. Example: CPU usage above 80%, DNS Concurrent requests crossing the pre-configured limit. Alert channels here can be Email, Slack, TEAMS, Discord, etc.
- Having a dashboard to monitor the alerts and improving the performance by tuning the application/system based on the alert reports.
- Analyzing the usage trends over performance test results and creating the right auto scaling group for backend services as required.
- Creating a dimensioning guide based on the resources consumed by the application/server/proxy. Why we need this? This will help in computing the right hardware size as required for the application. The system should not be under or over utilized.
- Setting up monitoring dashboard
- Exporting dashboard reports for a particular period or running a build to access the dashboard endpoint and export the results to a pdf file and upload it to an FTP or repository as required. These reports for a specific time range would help for future analysis. We could write NodeJS scripts using puppeteer to export the dashboards to PDFs.